computation part is done using JavaScript (JS). A JS file can be included either at the head or body of an HTML document. It contains functions that will implement the aforementioned operations. For drawing any content upon a canvas, a 2D rendering reference called context is supposed to be made.

Here's how to access images, as well as canvas and context, from JS:

```
//getting image, canvas and context
var im = document.getElementById("image_id");
var canvas = document.getElementById("canvas_id");
var context = canvas.getContext("2d");

//accessing a rectangular set of pixels through context
interface
var pixel = context.getImageData(x, y, width, height);

//displaying image data
context.putImageData(image, start_point_x, start_point_y);
```

Using a local Web cam from a browser

Accessing a Web cam from the browser first requires user consent. Local files with a URL pattern such as *file:///* are not allowed. Regular *https://* URLs are permitted to access media.

Whenever this feature is executed, the user's consent will be required. Any image or video captured by a camera is essentially media. Hence, there has to be a media object to set up, initialise and handle any data received by the Web cam. This ability of seamless integration is due to media APIs provided by the browser.

To access the Web cam with a media API, use this code:

```
navigator.getUserMedia = (
          navigator.getUserMedia ||
          navigator.webkitGetUserMedia ||
          navigator.mozGetUserMedia ||
          navigator.msGetUserMedia );
```

In the above code, *navigator.getUserMedia* will be set if the media exists. To get control of media (refers to camera), use the following code:

```
navigator.getUserMedia({
     video: true
},handle_video, report_error );
```

On the successful reception of a frame, the *handle_video* handler is called. In case of any error, *report_error* is called.

To display a frame, use the following code:

```
var video_frame = document.getElementById("myVideo");
video_frame.src = window.URL.createObjectURL(stream); //
stream is a default parameter
```

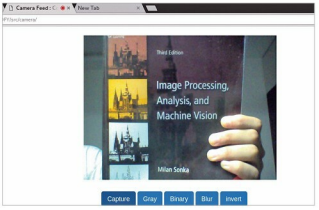

Figure 1: Displaying a frame

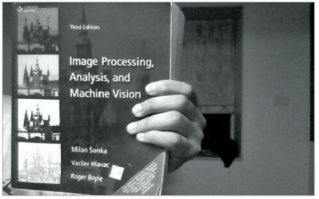

Figure 2: Grayscale image

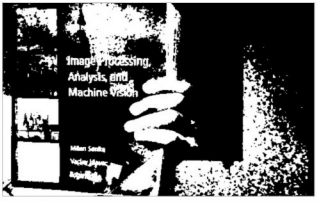

Figure 3: Binary image

```
//provided to handle_video
```

For further details, regarding a camera interfacing with the browser, refer to *https://github.com/kushalvyas/trackingjs_ofy/*.

The basic image processing algorithms

JS stores an image as a linear array in RGBA format. Each image can be split into its respective channels, as shown below:

```
var image = context.getImageData(0, 0, canvas.width, canvas.
height);
var channels == image.data.length/4;
```